

* Problem with Exposing Entities Directly :-

In standard Spring Boot apps, we often use the Entity class (which maps directly to database table) to receive requests & send responses. However, this is a bad practice because of :-

- Security Risks → When you expose an entity, a client can modify sensitive fields (e.g. deleted flag, createdAt, internal status) that should only be managed by server.
- Tight Coupling → Your API structure becomes bound to your database schema. If you decide to split a field (like firstName and lastName), your API contract breaks.
- Unnecessary Data Exposure → You might send internal data (like passwords or system flags) to clients that they don't need or shouldn't see.

* Solution :- DTOs (Data Transfer Objects) are plain Java classes used solely to transfer data between the client and the server. They act as a protective layer, allowing you to filter exactly what data is received and what is returned, effectively decoupling your Database Layer from your Client Interface.

Date.....

→ Request DTOs :- These contain only the fields the client is permitted to provide during a request (e.g. name, age, email, address, etc.). Sensitive fields like id, deleted flag are excluded.

→ Response DTOs :- These are used to send data back to client. They allow you to filter out internal server-side details (like database flags) and add user-friendly information (like a "success" message).

Since, entities and DTOs are separate, you must convert between them within your service layer:

→ Map to Entity :- When receiving a request, use a method (e.g. `mapToEntity`) to translate the DTO into Entity, while setting default server-side values (like `deleted = false` or `createdAt` timestamps).

→ Map to DTO :- After performing database operations, convert the resulting Entity back into a Response DTO using a `mapToDTO` method before returning it to the controller.

Date.....

* Even after using DTOs, you must ensure that the data sent by the client is valid. Instead of cluttering your Service Layer with manual validation logic, you can use Bean Validation to handle this directly at the Controller Layer.

→ Add "spring-boot-starter-validation" to your pom.xml.

→ Use validation annotations on your Request DTO fields to define rules:

- @NotBlank → Ensures a field is not null, empty, or just whitespace.
- @NotNull → Ensures the field is not null.
- @Size(min=x, max=y) → Restricts the length of strings or collections.
- @Min(x) → Sets a minimum value for numeric fields.

many more.